

IFT3355_Devoir3

Text Classification with scikit-learn

Contributors:

- Emanuel Rollin - 20106951

- Anne Sophie Rozéfort - 20189221

**** Voir Readme.md pour plus de details**

Part 1 - Classification avec Bag of Words et TFIDF

Q1.1 Quelle méthode (BoW ou TF-IDF) donne les meilleurs résultats ? Pourquoi pensez-vous que c'est le cas ?

This is the first data produced by our training. If it happens to be overwritten by an additional training, we can also refer to 'output.txt' to inspect the mean accuracy and the mean f1 scores.

Model	Accuracy BOW	Accuracy TF-IDF	F1 BOW	F1 TF-IDF
Linear Regression	0.981515	0.971644	0.927264	0.882551
Random Forest	0.978104	0.97631	0.911395	0.903437
MLP	0.983848	0.985462	0.936336	0.943601

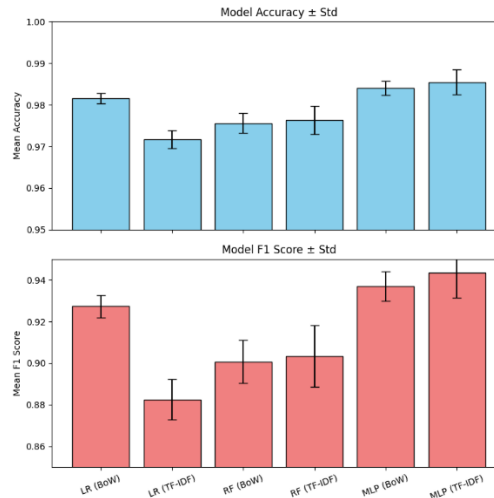
La précision est assez bonne parmi tous les modèles, qui marquent un message comme spam au moins 97% du temps.

On peut utiliser le F1 score pour comparer les modèles, qui considèrent aussi les . Ici, le MLP TF-IDF performe le mieux, ce qui semble prévisible avec l'idée qu'il s'agit d'un modèle plus puissant avec un filtre plus riche.

Aussi, en étudiant notre banque de données, on remarque que le vocabulaire est souvent très distinct entre un message SMS "ham" et "spam", car les "ham" utilisent des abréviations et des termes populaires, versus les "spam" mobilisent davantage de mots *normaux* ou de mots clés *corporate-esque*. Cela peut expliquer la bonne précision des modèles en général.

ham : "No..jst change tat only" spam : "You are guaranteed the latest Nokia
Phone, a 40GB iPod MP3 player or a 💎500 prize! Txt word: COLLECT to No:
83355!"

Il est intéressant de s'attarder sur la régression linéaire, qui semble préférer le prétraitement *Bag Of Words*. Dans l'article *geeksforgeeks.com* joint aux notes, il est discuté que la sensibilité du BOW est dictée par la largeur du document. TF-IDF normalise, ce qui est idéal pour les longs documents. Possiblement que pour les courts documents, comme pour des messages issus de la banque de messages SMS que nous avons dans 'spam.csv', BOW performe mieux tout simplement parce que nos données sont plus courtes comparativement à TF-IDF. La précision est bonne, ce qui signifie que lorsqu'on détecte quelque chose, c'est souvent bel et bien un spam. Le F1 score est moins bon, ce qui veut dire qu'on oublie davantage de vrais spams avec la régression linéaire.

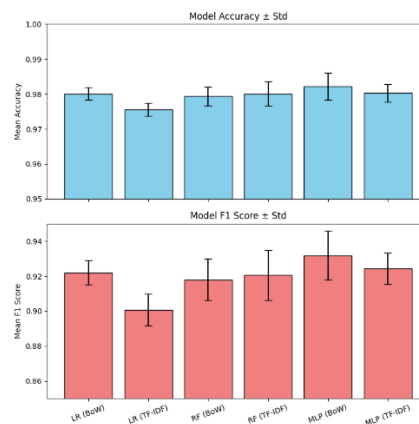


Q1.2 Que se passe-t-il pour les metriques si vous diminuez max features ? (Montrez un graphique)

Première conséquence : L'entraînement des modèles est fortement accéléré !

Model	Accuracy BOW	Accuracy TF-IDF	F1 BOW	F1 TF-IDF
Linear Regression	0.980079	0.975592	0.921957	0.900749
Random Forest	0.980078	0.978822	0.921126	0.91525
MLP	0.981874	0.980438	0.930265	0.924931

Ensuite, les métriques sont assez similaires avec un `max\_features` limité à 500 mots au lieu de 5000. Le modèle MLP avec TF-IDF semble celui le plus pénalisé, car il perd un peu de la richesse associée au TF-IDF qui valorise certains mots plus rares.

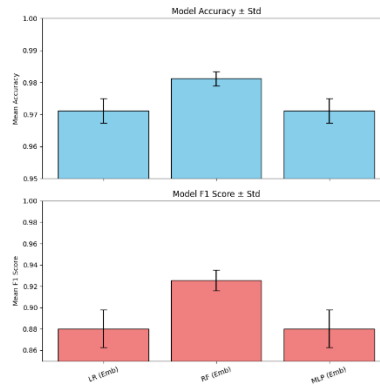


Part 2 - Classification avec Sentences Transformers

Q2.1 Comment les embeddings se comparent-ils à BoW et TF-IDF en termes de performance?

La méthode avec embeddings a en générale une précision plus basse sur les modèles, contrairement aux méthodes de BoW et TF-IDF, qui présentent une précision plus élevée soit d'environ 97%. De plus, le modèle obtient un meilleur score avec BoW et TD-IDF, surtout pour le MLP, tandis que pour les embeddings, c'est plutôt pour le modèle de

Random Forest que c'est le plus performant. De manière générale, la performance des embeddings restent inférieure à celle des deux autres méthodes. Les embeddings résument le texte dans un espace de taille fixe, ce qui fait en sorte que c'est possible de perdre le sens du texte ou des détails importants, surtout en classification binaire. Ce qui peut suggérer pourquoi les représentations basées sur les fréquences de mots comme BoW et TD-IDF bénéficient d'un meilleur vocabulaire et offre une meilleure performance, car ces dernières conservent une meilleure représentation des mots et capture mieux le texte. Donc, les méthodes BoW et TF-IDF surpassent la méthode d'embeddings en termes de précision. Dans ce contexte, les embeddings ont l'air d'être moins adaptés.



Q2.2 Quels sont les avantages et inconvénients d'utiliser des embeddings pré-entraînés ?

Les avantages du embeddings, c'est qu'on n'a pas besoin de beaucoup de données, ils peuvent quand même avoir un bon résultat avec un jeu de données limité. Les vecteurs sont aussi plus compacts ce qui peut améliorer l'efficacité du traitement. Cependant, ils peuvent perdre des bouts d'informations importantes à la classification, lorsque les informations sont compressées. Les embeddings sont aussi plus lents à utiliser surtout avec un codage dynamique tel qu'avec Sentence Transformers.

Part 3 - Le Super Learner

Q3.1

> Commenter les graphiques obtenus

Commentons brièvement sur la précision moyenne du superlearner (0.987975) et son score F1 moyen (0.954054) qui dépasse le meilleur modèle MLP utilisant TF-IDF seul (Accuracy 0.980438, F1 0.924931), ce qui implique davantage de détections et davantage de détections correctes.

Le résultat est un peu prévisible, mais on peut s'interroger au pourquoi est-ce que la précision augmente plutôt que de se comporter comme une moyenne, impliquant qu'ils ont un comportement complémentaire. Chaque modèle doit être sensible à un aspect différent des données. On sait que TF-IDF favorise l'extraction des sujets forts d'une phrase et l'identification des mots clés.

On sait déjà que le linear regressor va simplement se fier à une décision basée sur la somme des poids en fonction de leurs inputs. Le SVM entraîne son modèle en favorisant une séparation spatiale des données d'entraînement. Le MLP optimise sa descente du gradient.

Pour que leur contribution soit complémentaire et non nuisible (ex: LR nuit au modèle et affecte ses métriques à la baisse), la pondération est utilisée lors de l'entraînement pour donner un plus petit poids aux activations / détecteurs du LR, comme on peut le voir à la Q3.2.

Q3.2

>Quels sont les poids que votre méta-modèle a attribués à chaque modèle de base ?

Voici les poids associés à chaque modèle :

LR / SVM / MLP

[[2.05060466 4.78251414 6.29027211]]

On remarque l'importance accrue du MLP, ce qui est prévisible considérant sa bonne performance à la Q3.

